

# Kunai: Toward a Verifier-Safe Layered DSL for Nested Encapsulation Packet Filtering on eBPF

<sup>1, 2</sup>Takeru Hayasaka, <sup>1</sup>Satoshi Uda, <sup>3</sup>Ayako Hayasaka, <sup>4</sup>Daisuke Kotani

<sup>1</sup>japan Advanced Institute of Science and Technology, <sup>2</sup>SAKURA Internet Inc,

<sup>3</sup>Independent Researcher, <sup>4</sup>Kyoto University



# Problem: Capturing and storing every packet is costly!

---

- **Copy and store every packet:**

Copying all traffic uses CPU and memory bandwidth.[1]

Saving it also requires storage bandwidth and disk space.

- **Filter before userspace transfer:**

Filter in the kernel and send only the packets needed for the investigation to userspace.[2]

**=> Selection requires field reads!**

Read and compare fields whose positions depend on encapsulation, optional headers, and variable-length lists.

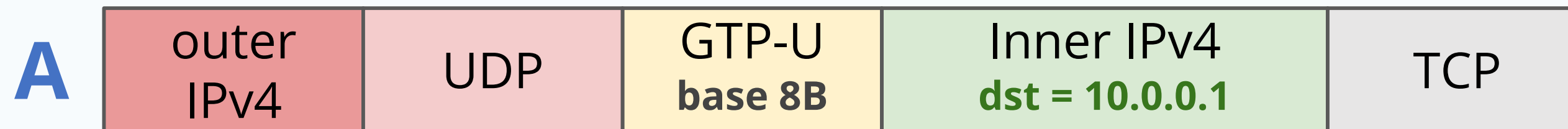
[1]: Comparing and Improving Current Packet Capturing Solutions based on Commodity Hardware, Lothar Braun et,al.(2010) <https://dl.acm.org/doi/10.1145/1879141.1879168>

[2]: Scap: stream-oriented network traffic capture and analysis for high-speed networks, Antonis Papadogiannakis et,al.(2013) <https://dl.acm.org/doi/abs/10.1145/2504730.2504750>

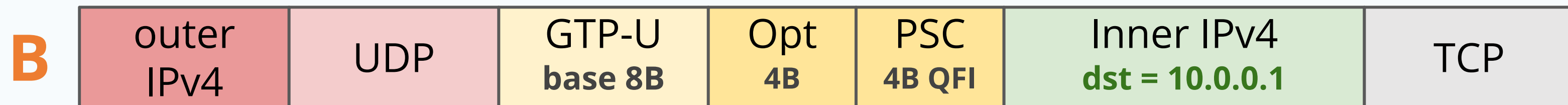
# Example: Read the destination IP of UE traffic

Investigate one SIM's session, Known UE (User Equipment) IPv4: 10.0.0.1  
Capture downlink TCP traffic to this UE inside GTP-U.

4G/LTE: Basic GTP-U (no optional fields)



5G: GTP-U with extension hdr (PSC: PDU Session Container, DL case)



⋮ → ⋮  
Inner IPv4 starts 8 B late

Monitoring LTE/5G roaming interfaces also requires handling packets with extensions\*

**=> Optional fields and extensions shift the inner IP header even when the destination stays the same!**

# Problem: Nested headers make fixed offsets hard to maintain

pcap-filter is the de facto capture-filter language used by tcpdump/libpcap.\*  
It has no named GTP-U inner fields, so these examples use hand-written offsets

## A: Basic GTP-U

```
udp dst port 2152 and  
udp[25] = 6 and  
udp[32:4] = 0x0a000001
```

## B: 5G with PSC

```
udp dst port 2152 and  
udp[33] = 6 and  
udp[40:4] = 0x0a000001
```

Offsets from UDP start, 6 == TCP , 0x0a000001==10.0.0.1

For these two layouts, A and B can be combined with "or" .

But changes to GTP-U extensions or other encapsulation headers can move the target fields

**=> Update the offset expressions whenever the target fields move...**

\*pcap-filter(7)man:<https://raw.githubusercontent.com/the-tcpdump-group/libpcap/libpcap-1.10.5/pcap-filter.manmisc.in>

# Proposal: Kunai, a DSL that computes field positions

---

GTP-U: Filter downlink TCP traffic to a specific UE at 10.0.0.1

```
eth/ipv4@outer/udp/gtp/ipv4@inner/tcp where inner.dst == 10.0.0.1
```

Same condition across the supported GTP-U layouts 🧑

SRv6: Filter packets whose segment list contains Router R's SID, fc00::1

```
eth/ipv6/srv6 where any(srv6.segments.addr == fc00::1)
```

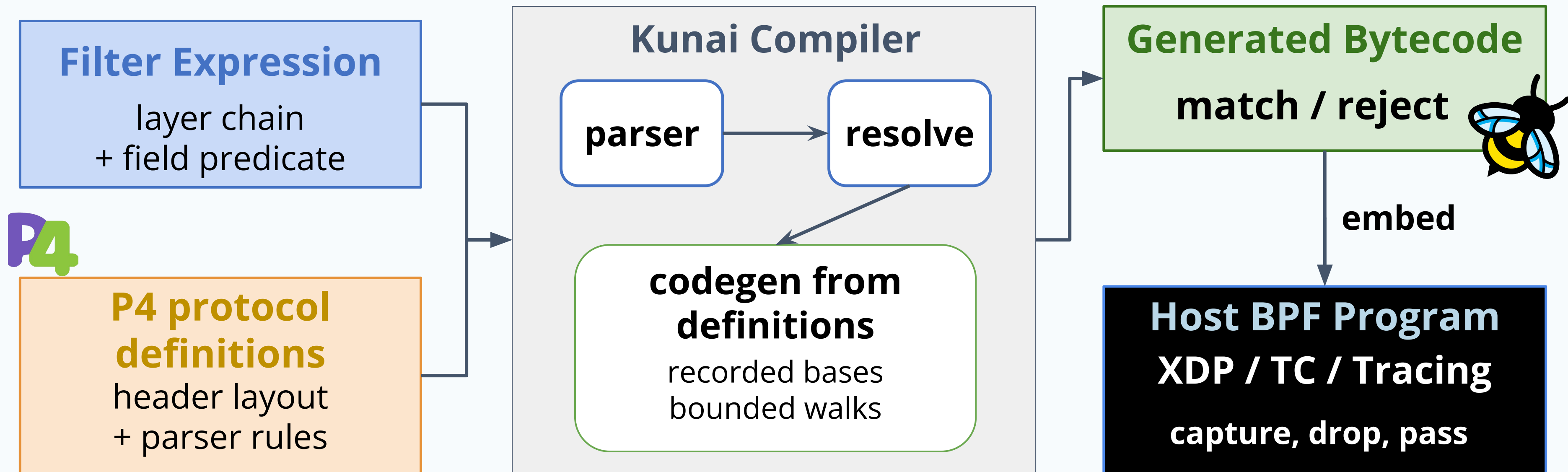
Match R's SID anywhere in the segment list that filtering 🧑

List A: [fc00::a, **fc00::1**]

List B: [fc00::a, fc00::b, **fc00::1**]

R SID is 2nd in A and 3rd in B.  
The same filter matches  
both packets!

# Kunai DSL: High-Level Architecture



**Kunai compiles a filter expression and P4 protocol definitions into eBPF bytecode that returns match or reject.** Embed the generated filter in XDP, tc, or tracing programs. The host provides packet access and handles the match result.

# Two challenges in implementing Kunai

---

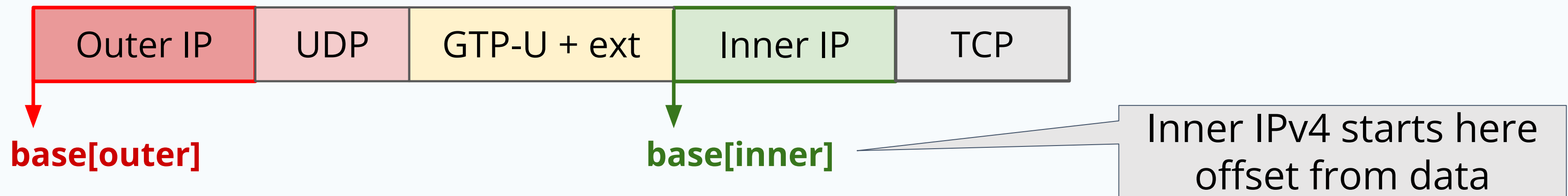
- **1. Generate safe reads at variable offsets**
  - The code must compute offsets from the packet and let the verifier establish read safety
  - => Generate checked reads and bounded walks
- **2. Derive walk rules from P4**
  - Code generation needs rules for field locations, protocol connections, and variable parts
  - => Extract rules from supported definitions

# Challenge 1: Locate and check each field

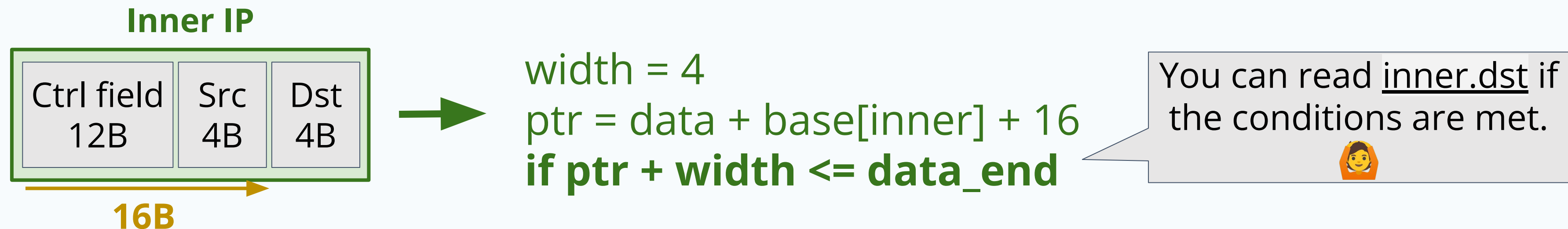
Consider safely reading the destination IP address from the inner IP header

```
eth/ipv4@outer/udp/gtp/ipv4@inner/tcp  
where inner.dst == 10.0.0.1
```

## 1 · RECORD HEADER START OFFSETS



## 2 · LOCATE THE FIELD AND CHECK THE READ



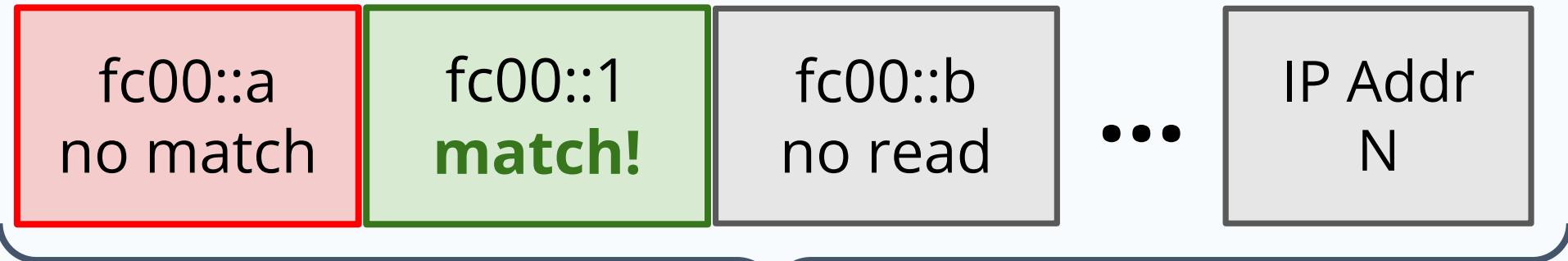
Kunai generates code that constrains offsets to a range the verifier can track and checks that the full field fits within packet bounds before reading it.

# Challenge 1: Walk until a match or a bound

Consider safely reading a SID at different positions in the segment list.

e.g. Filter by SID `fc00::1`

`eth/ipv6/srv6` where `any(srv6.segments.addr == fc00::1)`



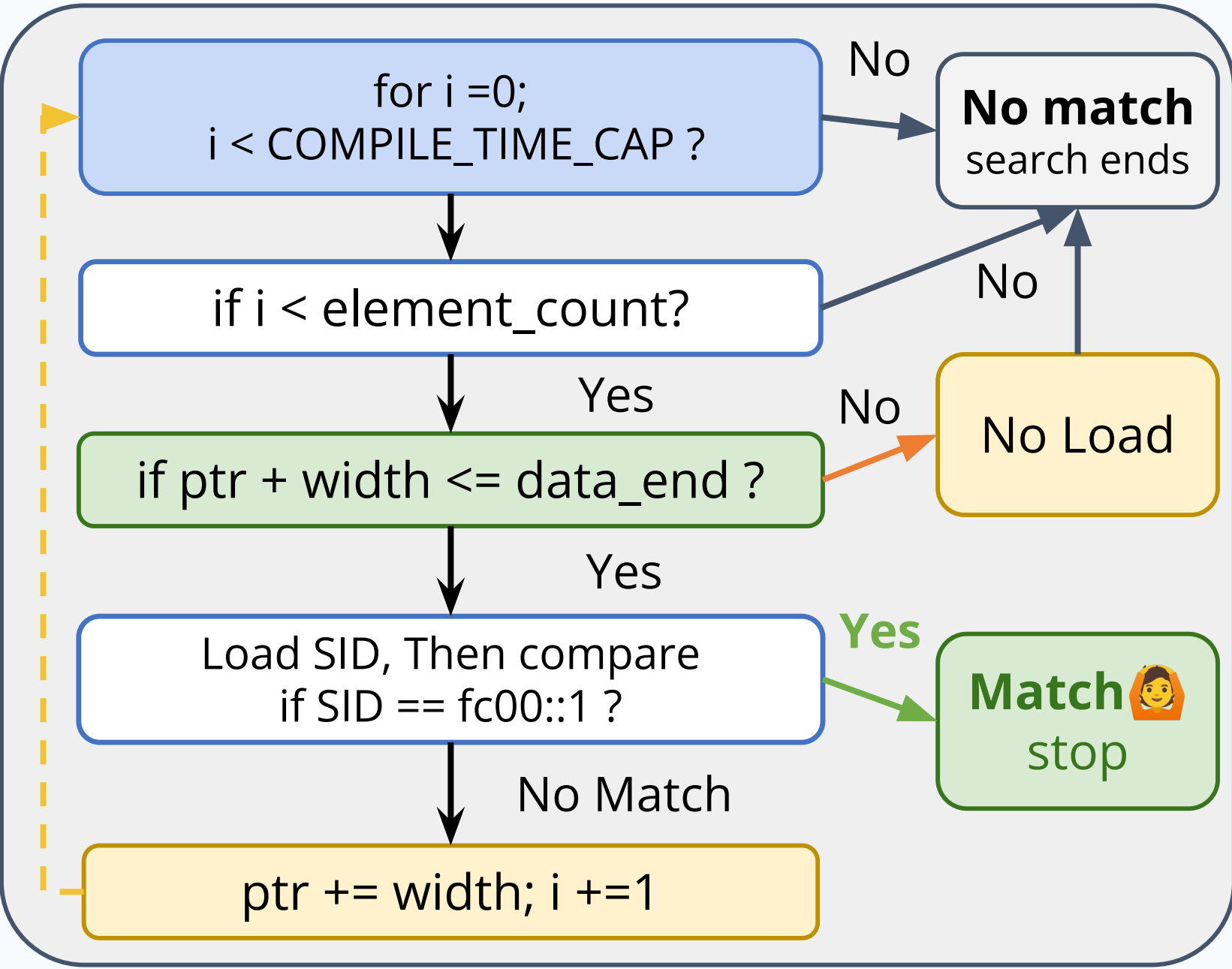
Element count (from packet)

```
width = 16 // ipv6 addr len
element_count = last_entry + 1
COMPILE_TIME_CAP = 8
```

Kunai unrolls the loop for caps of 1~2 and uses **bpf\_loop** for larger caps.  
The current P4 definition sets a cap of 8.

This walk needs the, **element width**, **element count**, and an **iteration cap**!

Conceptual walk



# Challenge 2: P4 layout, Protocol Connections, and walks

## Layout: header declarations

Relative field  
offsets  
and widths

## Protocol Connections

Conditions for  
entering  
the next layer  
(protocol num,  
port num...etc)

## Walks: Extraction, advance, transition

How to move  
through  
variable parts and  
stop

Example: connections in the GTP-U layer chain



# Challenge 2: P4 layout, Protocol Connections, and walks

**Layout:**  
header declarations  
udp.p4

```
header udp_h {  
  bit<16> sport;  
  bit<16> dport;  
  bit<16> length;  
  bit<16> checksum;  
}
```

**Protocol Connections**  
udp.p4 + gtp.p4

```
const bit<8>  
KUNAI_UDP_IPV4_PROTOCOL  
= 17;  
const bit<16>  
KUNAI_GTP_UDP_DPORT  
= 2152;
```

**Walks: Extraction, advance, transition**  
gtp.p4

```
state parse_ext {  
  pkt.extract(exts.next);  
  transition select(  
    exts.last.next_ext) {  
    0: accept;  
    _: parse_ext;  
  }  
}
```

Example: connections in the GTP-U layer chain



# Challenge 2 Example: The SRv6 parser walk

```
const bit<8> KUNAI_SRV6_IPV6_NEXT_HEADER = 43; // IPv6 next_header -> IPv6 Routing Header
const bit<8> SRV6_ROUTING_TYPE = 4;           // routing_type identifies SRH
```

Walk details:

Array Capacity

Elem count /termination

Extract one elem/decrement

## Declarations and Init

```
parser SRv6Parser(packet_in pkt,
  out srv6_h hdr,
  out srv6_seg_h[8] segments) {
  ParserCounter() pc;

  state start {
    pkt.extract(hdr);
    pc.set((bit<8>)(hdr.last_entry + 1));
    transition select(hdr.routing_type) {
      SRV6_ROUTING_TYPE: walk;
      default: reject;
    }
  }
```

## Loop states (left code continued)

```
state walk {
  transition select(pc.is_zero()) {
    true: accept;
    false: consume_seg;
  }
}

state consume_seg {
  pkt.extract(segments.next);
  pc.decrement(1);
  transition walk;
}
```

start => [walk ⇌ consume\_seg] (loop) if counter reaches zero => accept (parse done!)

# Challenge 2: Derive walk rules from P4

Recognize a counter, repeated extraction, and decrement in the parser's states.

## P4 code Excerpt (states omitted)

```
out srv6_seg_h[8] segments
```

```
pc.set((bit<8>)(hdr.last_entry + 1));
```

```
// one 16-byte segment  
pkt.extract(segments.next);  
pc.decrement(1);
```

## Derived walk parameters

**Compile-Time Cap**  
8 (array capacity)

**Element count (from Packet)**  
last\_entry + 1 (e.g. 2 + 1 = 3 elem)

**Per Iteration**  
Extract 1 element  
(decrement counter by 1)

# Evaluation Step

---

- **1. Expressiveness:** 10 target filters
  - F1-F10 vs. pcap-filter syntax
- **2. Loading and Match correctness:** 77 corpus expressions
  - A separate language-feature set
  - Loading tested on six Linux versions  
(6.1 / 6.6 / 6.12 / 6.15 / 6.18 / 7.0)
- **3. Processing-rate cost:** 10 target filters
  - Added cost vs. a same-copy baseline

# 1. Expressiveness: Header and field conditions without manual offsets

|         | Target                                                  | Kunai   | pcap-filter |
|---------|---------------------------------------------------------|---------|-------------|
| F1 - F6 | basic headers + QinQ<br>e.g. F1: IPv4/TCP, dst port 443 | 6 / 6   | 6 / 6       |
| F7      | GTP-U -> Inner IPv4                                     | ✓       | —           |
| F8      | any SRv6 segment                                        | ✓       | —           |
| F9      | Geneve -> inner IPv4                                    | ✓       | —           |
| F10     | TCP MSS Option                                          | ✓       | —           |
|         |                                                         | 10 / 10 | 6 / 10      |

Comparison: Structured header and field notation.

Kunai supports conventional filters and nested-packet conditions without hand-written offsets for each supported layout.

# 2. Loading and Match correctness: All checks passed

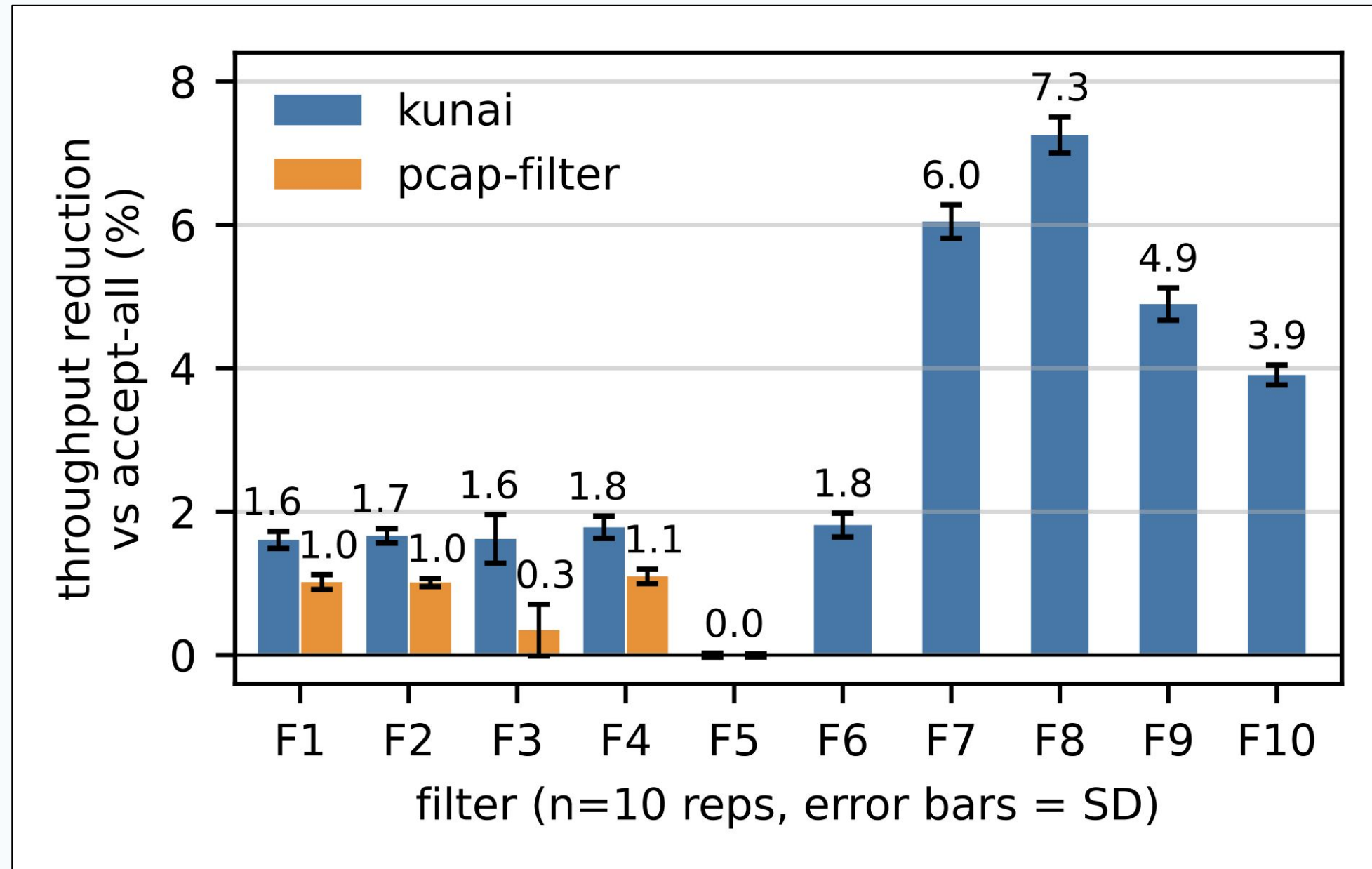
| check                             | Passed / Tested | Scope within the 77 expr corpus                                     |
|-----------------------------------|-----------------|---------------------------------------------------------------------|
| XDP load                          | 77 / 77         | All expressions                                                     |
| TC load                           | 74 / 74         | 74 = 77 - 3<br>3 is unsupported VLAN/QinQ forms by kunai tc backend |
| Match correctness (XDP test exec) | 74 / 74         | 74 = 77 - 3<br>3 is capture clause expressions*                     |

- **Loading:** Check whether the verifier accepts the generated eBPF. (on 6 Linux versions\*\*)
- **Match correctness:** Check whether test packets match as expected.

`eth/ipv4/tcp capture headers+64`  
`eth/ipv4/tcp capture all`  
`eth/ipv4/tcp capture absolute 96`

The **capture** clause sets the output length.  
All three share `eth/ipv4/tcp`  
We test the shared base expression as one of the 74.

### 3. Processing-rate cost: $\leq 7.3\%$ rate reduction



Condition Note: Rate measurement environment, Linux 6.15, BPF JIT, native XDP\_DROP, Xeon 8362 · Intel E810 · 100 GbE, 64-byte line-rate load · 10 repetitions

**These results show that Kunai can filter different packet structures in the kernel !!**

#### With and without filter evaluation

Both programs copy the same bytes and drop every packet.  
Filter evaluation is added between these steps.

#### Shared filters · F1-F5

Within **1.3 pp of pcap-filter** in processing-rate reduction.

#### Complex filters · F7-F10

GTP-U, SRv6, Geneve, TCP options

**3.9- 7.3% rate reduction**

vs. the baseline without filter evaluation

# Limitations and Related work

---

## Limitations

### Implementation gaps

Example: GTP extension headers over 4 bytes. The current definition reads 4 bytes per entry.

### Packet filtering

Stateful filtering, deep packet inspection, and aggregation are outside the scope.

### Empirical verifier acceptance

No formal guarantee for every expression or kernel version

## Related work

### NetPDL / NetPFL [1,2]

Protocol descriptions and tunnel-aware filter expressions

### P4C-XDP [3]

P4 programs => eBPF for XDP

### Kunai

Filter expressions P4 definitions  
=> eBPF with checked reads  
and bounded walks.

[1]: NetPDL: An extensible XML-based language for packet header description, Fulvio Risso et al (2006), <https://doi.org/10.1016/j.comnet.2005.05.029>

[2]: A Tunnel-Aware Language for Network Packet Filtering, Luigi Ciminiera et al (2010), <https://ieeexplore.ieee.org/document/5683161/>

[3]: P4C-XDP: Programming the Linux Kernel Forwarding Plane Using P4, Fabian Ruffy et al (2018), <https://lpc.events/event/2/contributions/97/>

# Conclusion: Specify fields and conditions with Kunai

---

**Proposal** A DSL for inner-header and list conditions, without hand-written offsets for each supported layout.

---

**Mechanism** Kunai generates checked field reads and bounded traversal from P4 layouts, connections, and walk rules.

---

**Evidence** **Expressiveness:** Complex filter conditions without manual offsets  
**Checks:** XDP loads 77 / 77 · XDP match tests 74 / 74.  
**Rate cost:** up to 7.3% rate reduction from adding filter evaluation

---

**Example:**  
UE traffic inside  
GTP-U

```
eth/ipv4@outer/udp/gtp/ipv4@inner/tcp  
where inner.dst == 10.0.0.1
```

# Appendix

# What the evaluation showed

---

## 1. Expressiveness

Inner-header and list conditions were expressed with field names, without manual byte offsets.

## 2. Loading and match correctness

Generated eBPF passed load checks and selected packets as expected

## 3. Processing cost

Common filters (F1–F5): within 1.3 pp of pcap-filter in processing-rate reduction.  
Complex filters (F7–F10): 3.9–7.3% rate reduction vs. the same-copy baseline without filter evaluation

**These results show that Kunai can filter different packet structures in the kernel**

# Program size vs. verifier work

---

| Filter                           | Program instructions | Instructions processed by verifier (max) |
|----------------------------------|----------------------|------------------------------------------|
| <b>F8 · SRv6 segment walk</b>    | 333                  | 3,987                                    |
| <b>F10 · TCP MSS option walk</b> | 231                  | 206,733                                  |

- Verifier work depends on explored paths and the kernel version.
- To compare loop strategies, run the same filter with each implementation

# Other supported traversal patterns

---

| Structure                                    | Definition supplies                    | Walk uses                                                     |
|----------------------------------------------|----------------------------------------|---------------------------------------------------------------|
| <b>SRv6 segment list</b><br>Counted elements | Counter +<br>repeated extraction       | Element count (from packet)<br>fixed 16B stride               |
| <b>TCP options</b><br>Kind-dispatched loop   | Kind dispatch +<br>extraction / length | Advance by option size.<br>stop at EOL or the iteration limit |
| <b>MPLS stack</b><br>Header chain            | Declared bottomofstack<br>marker       | End marker + quantifier bound                                 |

# TCP options 1: Dispatch by kind in P4

```
const bit<8> KUNAI_TCP_IPV4_PROTOCOL = 6;  
const bit<8> KUNAI_TCP_IPV6_NEXT_HEADER = 6;
```

```
parser TcpParser(packet_in pkt,  
out tcp_h hdr, out tcp_opt_mss_h mss, ...)
```

Walk details:

Layout / size

Kind / termination

Extract / advance

## Start and kind dispatch

```
state start {  
  pkt.extract(hdr);  
  transition select(hdr.data_offset) {  
    5: accept; default: parse_options;  
  }  
}  
  
state parse_options {  
  transition select(pkt.lookahead<bit<8>>()) {  
    0: accept; // EOL  
    1: parse_nop;  
    2: parse_mss;  
    3: parse_ws; 4: parse_sack_perm;  
    5: parse_sack; 8: parse_ts;  
    default: parse_unknown_opt;  
  }  
}
```

## MSS fields and option handlers

```
header tcp_opt_mss_h {  
  bit<8> kind;  
  bit<8> length;  
  bit<16> value;  
}  
  
state parse_nop { pkt.advance(8);  
  transition parse_options;  
}  
  
state parse_mss { pkt.extract(mss);  
  transition parse_options;  
}  
  
state parse_unknown_opt {  
  pkt.advance(((bit<32>  
    pkt.lookahead<bit<16>>()[7:0]) << 3);  
  transition parse_options;  
}
```

# TCP options 2: Locate the MSS value

**eth/ipv4/tcp** where **tcp.options.mss.value == 1460**

| Rule                       | Definition and generated behavior                                                                           |
|----------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>Identify MSS</b>        | Kind Type2 selects parse_mss.                                                                               |
| <b>Advance options</b>     | NOP: 1B, MSS: 4B from its header type.<br>Unknown options: advance by the Length field.                     |
| <b>Record the position</b> | Record the MSS start and continue parsing.                                                                  |
| <b>Read and compare</b>    | After parsing, read 2B at MSS start + 2.<br>The value follows Kind (1B) and Length (1B). Compare with 1460. |
| <b>Code generation</b>     | Check reads and advances; cap the walk at 8 iterations by default.                                          |

# MPLS 1: Declare the header and chain end

Walk details:

Layout / default bound

End marker

Extract / repeat connection

## One entry and its parse

```
header mpls_h {  
  bit<20> label;  
  bit<3> tc;  
  bit<1> s;  
  bit<8> ttl;  
}  
parser MplsParser(packet_in pkt,  
  out mpls_h hdr) {  
  state start {  
    pkt.extract(hdr);  
    transition accept;  
  }  
}
```

## Connections, default depth, and end marker

```
const bit<16>  
  KUNAI_MPLS_ETH_ETHERTYPE = 0x8847;  
const bool  
  KUNAI_MPLS_MPLS_NO_CHECK = true;  
const bit<8>  
  MPLS_MAX_DEPTH = 8;  
const bit<1>  
  MPLS_CHAIN_END_S = 1;
```

One entry is 32 bits = **4 bytes**. s = 1 marks the last entry.  
The parser reads one entry. The DSL requests repetition, for example mpls{1,4} .

# MPLS 2: Walk to the end bit within a bound

**eth/mpls{1,4}/ipv4/tcp**

| Rule                | From the definition and DSL                                                      |
|---------------------|----------------------------------------------------------------------------------|
| Enter from Ethernet | Check <code>ethertype = 0x8847</code>                                            |
| Advance one entry   | 32 bits in <code>mpls_h</code> => 4B per entry                                   |
| End of the stack    | Consumed entry has <code>s = 1</code> => continue to IPv                         |
| Read and compare    | <code>{1,4}</code> => 1-4 entries reject if entry 4 still has <code>s = 0</code> |